

Reviewer Pack — Minimal Local Index in Algebraic Topology and Resolution Pro...

Entry c083b6ec138c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 01:12:28 UTC

Minimal Local Index in Algebraic Topology and Resolution Proof Width Bound

Entry ID: c083b6ec138c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 01:12:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every 3-colorable graph G , the minimal local index of its associated simplicial complex is linearly correlated with its resolution proof width $w(G)$, such that $\min_local_index(G) = \Theta(w(G))$.

Rationale (proposer's reasoning):

Algebraic topology provides a rich framework for studying the connectivity and structure of geometric objects. The local index measures the complexity of the simplicial complex associated with a graph, which can be used to understand the difficulty of finding a proof in resolution. A strong correlation between these two quantities could reveal a fundamental connection between topological properties and computational complexity.

Taxonomy category: Algebraic Topology (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cf2bc29b562adb8e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every 3-colorable graph G with $n \leq 40$ vertices, if the correlation coefficient between $\min_local_index(G)$ and $w(G)$ is greater than or equal to 0.7 over 30 random seeds, then the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (algebraic topology AND local systems) AND (Boolean satisfiability OR resolution proof complexity) - minimal local index IN algebraic topology AND resolution proof width - (3-colorable graph AND simplicial complex) AND (resolution proof complexity OR Boolean satisfiability)

Top relevant hits considered: - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/1607.00443v1>] Algebraic Proof Complexity: Progress, Frontiers and Challenges - [<http://arxiv.org/abs/2602.24164v1>] Complexity of Satisfiability in Kochen-Specker Partial Boolean Algebras - [<http://arxiv.org/abs/1002.3702v2>] Algebraic K-theory over the infinite dihedral group: a controlled topology approach - [<http://arxiv.org/abs/math/0307111>] Extended Bloch group and the Cheeger-Chern-Simons class - [<http://arxiv.org/abs/1108.4976v2>] Cyclic A_∞ -Structures and Deligne's Conjecture - [<http://arxiv.org/abs/2206.13463v4>] Line graphs of simplicial complexes - [<http://arxiv.org/abs/2506.12921v2>] Shortest Paths in a Weighted Simplicial Complex

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3colorable_graph(n):
        if n < 5 or n > 40:
            return None, None

        graph = [[] for _ in range(n)]
        colors = [None] * n

        # Ensure the graph is 3-colorable
        for i in range(n):
            available_colors = set(range(3))
            for j in range(i):
                if j in graph[i]:
                    available_colors.discard(colors[j])
            color = random.choice(list(available_colors))
            colors[i] = color

        # Add edges to ensure the graph is 3-colorable
        for j in range(n):
```

```

        if j != i and colors[j] != color:
            graph[i].append(j)
            graph[j].append(i)

    return graph, colors

def compute_min_local_index(graph):
    n = len(graph)
    simplicial_complex = []

    # Construct the simplicial complex
    for i in range(n):
        for j in range(i + 1, n):
            if j in graph[i]:
                simplicial_complex.append((i, j))

    # Compute the minimal local index
    min_index = float('inf')
    for simplex in simplicial_complex:
        neighbors = set()
        for v in simplex:
            neighbors.update(graph[v])
        min_index = min(min_index, len(neighbors) - len(simplex) + 1)

    return min_index

def compute_resolution_width(graph):
    n = len(graph)
    clauses = []

    # Create clauses for each vertex
    for i in range(n):
        clause = [j for j in range(n) if j != i]
        clauses.append(clause)

    # Resolve the clauses using a small DPLL solver
    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            if literal < 0:
                literal = -literal
                value = False
            else:
                value = True
            assignment[literal] = value
            new_clauses = [c for c in clauses if literal not in c and -literal not in c]
            return dpll(new_clauses, assignment)
        pure_literal = next((l for l in range(1, n + 1) if (l not in assignment and -l not in assignment)), None)
        if pure_literal:
            value = True
            assignment[pure_literal] = value
            new_clauses = [c for c in clauses if pure_literal not in c and -pure_literal not in c]
            return dpll(new_clauses, assignment)

```

```

        literal = random.choice(range(1, n + 1))
        value = True
        assignment[literal] = value
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        if dpll(new_clauses, assignment):
            return True
        del assignment[literal]
        value = False
        assignment[literal] = value
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        if dpll(new_clauses, assignment):
            return True
        del assignment[literal]
        return False

assignment = {}
if not dpll(clauses, assignment):
    return 0

# Compute the resolution width
max_width = 0
for literal in assignment:
    if assignment[literal]:
        width = sum(1 for c in clauses if literal in c)
        max_width = max(max_width, width)

return max_width

graph, colors = generate_3colorable_graph(random.randint(5, 40))
if not graph or not colors:
    return {
        "metric_name": "min_local_index",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "graph_not_3colorable"
    }

min_local_index = compute_min_local_index(graph)
resolution_width = compute_resolution_width(graph)

return {
    "metric_name": "min_local_index",
    "metric_value": min_local_index,
    "instances_tested": 1,
    "n_max": len(graph),
    "conjecture_holds": min_local_index >= 0.7 * resolution_width,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None)) ** 0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"not_supported\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02bc0617.py", line 152, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02bc0617.py", line 123, in run_trial
    graph, colors = generate_3colorable_graph(random.randint(5, 40))
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02bc0617.py", line 34, in generate_3colorable_graph
    color = random.choice(list(available_colors))
                    ~~~~~
  File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
IndexError: Cannot choose from an empty sequence

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to calculate the correlation coefficient. | next: Review and debug the test code to ensure it can run to completion without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17005
2	propose	ollama_remote	glm4:latest	0	0	11812
3	preregistration	ollama_remote	glm4:latest	0	0	9403
4	novelty	ollama_remote	glm4:latest	0	0	8559
5	novelty	ollama_remote	glm4:latest	0	0	18216
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29513
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15804
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13613
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15744
10	judge	ollama_remote	glm4:latest	0	0	23113

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162781 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c083b6ec138c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c083b6ec138c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c083b6ec138c.tar.gz` (if generated)